

C18 – The STL list - Practice Problems (Solution Key)

Problem 1: Task Scheduler with In-Place Splicing (std::list)

Problem: Implement a task queue using `std::list`. Write a function `prioritizeTask` that moves a specific task (pointed to by an iterator) to the front of the list in $O(1)$ time using `splice`.

Solution:

```
#include "utility.h"

// Solution 1: Moves a task to the head in O(1) using splice
void prioritizeTask(list<string>& tasks, list<string>::iterator it) {
    // list<string>::splice(iterator position, list<string>& other, iterator it)
    // Moves 'it' to 'position' within the same list<string> without copying nodes.
    tasks.splice(tasks.begin(), tasks, it);
}

// Example Usage:
int main() {
    list<string> scheduler = { "Compile", "Link", "Run", "Debug" };

    // Prioritize the task "Run" (iterator pointing to the 3rd element)
    auto it = next(scheduler.begin(), 2);
    prioritizeTask(scheduler, it);

    cout << "Scheduled tasks: ";
    for (const auto& task : scheduler) cout << "[" << task << " ] ";
    // Output: Scheduled tasks: [Run] [Compile] [Link] [Debug]
}
```

Problem 2: Remove Blacklisted Elements (std::list & std::set)

Problem: Given a `std::list` containing duplicate elements, remove all occurrences of numbers that also exist in a provided `std::set blacklist`.

Solution:

```
#include "utility.h"

//=====

// Version 1: Uses remove_if with lambda for optimal performance
void removeBlacklisted(list<int>& lstCandidates, const set<int>& setBlacklist) {
    for (auto sitem : setBlacklist) {
        auto it = find(lstCandidates.begin(), lstCandidates.end(), sitem);
        if (it != lstCandidates.end()) lstCandidates.erase(it);
    }
}

/* *****

// Version 2: Uses remove_if with lambda for optimal performance
void removeBlacklisted(list<int>& candidates, const set<int>& blacklist) {
    // list::remove_if(predicate) deletes nodes that satisfy the condition.
    // capture clause [&] takes by reference the caller's context
    candidates.remove_if([&](int val) {
        return blacklist.find(val) != blacklist.end();
    });
}

*****/

// Example Usage:
int main() {
    list<int> lstCandidates { 11, 33, 55, 77, 99 };
    set<int> setDisqualified { 33, 99 };

    showList(lstCandidates, "lstCandidates before removal");
    removeBlacklisted(lstCandidates, setDisqualified);
    showList(lstCandidates, "lstCandidates after removal ");
}
```

Problem 3: Merging Sorted Execution Logs (std::list::merge)

Problem: Merge two sorted execution log streams (`std::list`) ordered by timestamp into a single sorted list. Do this without extra copies or re-sorting using `list::merge`.

Solution

```
#include "utility.h"

struct LogEntry {
    long    timestamp;
    string  message;

    friend bool operator< (const LogEntry& le1, const LogEntry& le2) {
        return le1.timestamp < le2.timestamp;
    }

    friend ostream& operator<< (ostream& sout, const LogEntry& le) {
        sout << "[" << le.timestamp << ", "
              << le.message << "];"
        return sout;
    }
};

//Version 1: Manually merge sorted lists
void mergeLogs(list<LogEntry>& log1, const list<LogEntry>& log2) {

    auto it1 = log1.begin();
    auto it2 = log2.begin();

    while (it1 != log1.end() && it2 != log2.end()) {
        if (*it1 < *it2) {
            it1++;
        }
        else { //copy data from log2 to log1
            log1.insert(it1, *it2);
            it2++;
        }
    }

    // Append any remaining log2 elements that are larger
    // than all elements in log1
    while (it2 != log2.end()) {
        log1.push_back(*it2);
        ++it2;
    }
}
```

```

/* *****
// version2: Merges log2 into log1 in sorted order using merge
void mergeLogs(list<LogEntry>& log1, list<LogEntry>& log2) {
    // std::list::merge(list& other, Compare comp)
    // Merges sorted 'other' into sorted '*this' based on comp.
    log1.merge(log2, [](const LogEntry& a, const LogEntry& b) {
        return a.timestamp < b.timestamp;
    });
}
*****/
int main() {
    list<LogEntry> le1{
        {100, "msg100"},
        {200, "msg200"},
        {300, "msg300"}
    };
    list<LogEntry> le2{
        {150, "msg150"},
        {170, "msg170"},
        {250, "msg250"},
        {300, "msg300x"},
        {400, "msg400"},
    };
    mergeLogs(le1, le2);
    showList(le1, "After merge", true);
}

```

After merge (size=8)

```

-----
[100, msg100]
[150, msg150]
[170, msg170]
[200, msg200]
[250, msg250]
[300, msg300x]
[300, msg300]
[400, msg400]

```

Problem 4: Partitioning Vector to Dual Lists (std::vector & std::list)

Problem: Convert a `std::vector` into a `std::list`. Then, partition the numbers into two lists: one for positives and one for negatives.

Solution

```
#include "utility.h"

// Distributes vector values into two lists
void partitionVector(const vector<int>& vSource, list<int>& vpos, list<int>& vneg) {
    for (double val : vSource) {
        if (val >= 0.0) {
            vpos.push_back(val);
        }
        else {
            vneg.push_back(val);
        }
    }
}

int main() {
    vector<int> v{ -1, 2, -3, 4, 5, -3, 6, 7, -10 };
    list<int> lstPos;
    list<int> lstNeg;
    partitionVector(v, lstPos, lstNeg);
    showList(lstPos, "List of positive numbers");
    showList(lstNeg, "List of negative numbers");
}
```

Problem 5: Text Editor Cursor Operations (std::list)

Problem: Model a simple text editor buffer. Write a function to insert a character at the current cursor position and advance the cursor.

Sample Output

```
CS2 Assignment: Text Editor lstBuffer Solution

lstBuffer: | [^] |

--- Stage 1: Basic Typing "CS202" ---
Typed 'C' -> lstBuffer: | C [^] |
Typed 'S' -> lstBuffer: | C S [^] |
Typed '2' -> lstBuffer: | C S 2 [^] |
Typed '0' -> lstBuffer: | C S 2 0 [^] |
Typed '2' -> lstBuffer: | C S 2 0 2 [^] |

--- Stage 2: itCursor Navigation & Mid-Insert ---
Moving itCursor left 3 positions... -> lstBuffer: | C S [^]2 0 2 |
Moving itCursor right 1 position... -> lstBuffer: | C S 2 [^]0 2 |
Typed: 'x' -> lstBuffer: | C S 2 x [^]0 2 |
Typed: 'y' -> lstBuffer: | C S 2 x y [^]0 2 |

All done!
```

Solution

```
#include "utility.h"

// =====
// Typographically Correct Verbatim C++ Solution for Problem 6
// Text Editor lstBuffer using list with an Iterator itCursor
// =====

// Inserts a character before the itCursor position.

void insertChar(list<char>& lstBuffer, list<char>::iterator& itCursor, char c) {
    // Inserts c BEFORE itCursor. The itCursor remains on the *same* element,
    // which effectively moves it one position AFTER the newly inserted character.
    lstBuffer.insert(itCursor, c);
}

// Basic itCursor Movement Function: Moves itCursor left.
void moveCursorLeft(const list<char>& lstBuffer, list<char>::iterator& itCursor) {
    // Cannot move left if the itCursor is already at the beginning of the lstBuffer.
    if (itCursor != lstBuffer.begin()) {
        --itCursor;
    }
}
```

```

// Basic itCursor Movement Function: Moves itCursor right.
void moveCursorRight(const list<char>& lstBuffer, list<char>::iterator& itCursor) {
    // Cannot move right if the itCursor is already at the end of the lstBuffer
    if (itCursor != lstBuffer.end()) {
        ++itCursor;
    }
}

// Helper function to display the lstBuffer's contents with a itCursor symbol [^].
void displayBufferAndCursor(const list<char>& lstBuffer,
                           const list<char>::iterator& itCursor) {
    cout << "lstBuffer: | ";
    for (auto it = lstBuffer.begin(); it != lstBuffer.end(); ++it) {
        if (it == itCursor) {
            cout << "[^]"; // itCursor representation: [^]
        }
        cout << *it << " ";
    }

    // Special check: Is the itCursor pointing to the end sentinel?
    if (itCursor == lstBuffer.end()) {
        cout << "[^]"; // itCursor at end
    }
    cout << " |" << endl;
}

// =====
// Main Execution / Testing for Problem 6 (Simulating Typing)
// =====
int main() {
    cout << "CS2 Assignment: Text Editor lstBuffer Solution \n\n";

    // Declare the list. Set itCursor to the beginning of the list
    list<char> lstBuffer;
    auto itCursor = lstBuffer.begin();

    displayBufferAndCursor(lstBuffer, itCursor);

    // Test Sequence 1: Inserting characters "CS202"
    cout << "\n--- Stage 1: Basic Typing \"CS202\" ---\n";
    string keys_pressed = "CS202";
    for (char c : keys_pressed) {
        insertChar(lstBuffer, itCursor, c);
        cout << "Typed '" << c << "' -> ";
        displayBufferAndCursor(lstBuffer, itCursor);
    }
    // Expected Result: lstBuffer: | C S 2 0 2 [^] | (itCursor at end)

    cout << "\n--- Stage 2: itCursor Navigation & Mid-Insert ---\n";
    // Test Case 2a: Move Left (between 'S' and '2')
    cout << "Moving itCursor left 3 positions... -> ";
    for (int i = 0; i < 3; ++i) moveCursorLeft(lstBuffer, itCursor);
    displayBufferAndCursor(lstBuffer, itCursor);
    // Expected Result: lstBuffer: | C S [^]2 0 2 |

    // Test Case 2b: Move Right once (between '2' and '0')
    cout << "Moving itCursor right 1 position... -> ";
    moveCursorRight(lstBuffer, itCursor);
    displayBufferAndCursor(lstBuffer, itCursor);
    // Expected Result: lstBuffer: | C S 2 [^]0 2 |

```

```

// Test Case 2c: Insert 'x'
char mid_key = 'x';
insertChar(lstBuffer, itCursor, mid_key);
cout << "Typed: " << mid_key << " -> ";
displayBufferAndCursor(lstBuffer, itCursor);
// Expected Result: lstBuffer: | C S 2 x [^]0 2 |

// Test Case 2d: Insert 'y'
char mid_key_2 = 'y';
insertChar(lstBuffer, itCursor, mid_key_2);
cout << "Typed: " << mid_key_2 << " -> ";
displayBufferAndCursor(lstBuffer, itCursor);
// Expected Result: lstBuffer: | C S 2 x y [^]0 2 |

cout << "\nAll done!\n";

}

```

Problem 6: Fast access to a list using a map<>

Problem: Springfield maintains a database of its employees. Each employee has a unique employee ID (`eid`) and a name (`ename`).

For example:

```
101 Homer Simpson
102 Marge Simpson
103 Bart Simpson
104 Lisa Simpson
105 Ned Flanders
```

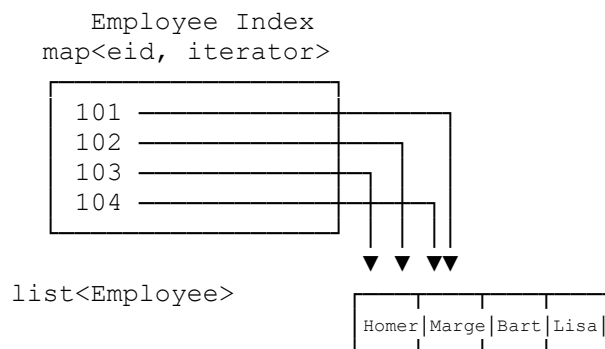
The employee records must be stored in a `list<Employee>`. The list is the **primary data structure** containing the actual employee objects.

To make it possible to find an employee efficiently by employee ID, create a `map` that acts as an **index**. The map must use the employee ID as its key and store an **iterator pointing to the corresponding employee in the list**.

Use the following relationship:

```
map<int, list<Employee>::iterator>
```

Conceptually:



The list owns the `Employee` objects.
The map is an index into the list.

Requirements

1. Define an `Employee` class/struct containing:
 - o `int eid`
 - o `string ename`
 - o Overload operators `<<`, `and` `<`

2. Store the employees in:

```
list<Employee> employees;
```

3. Create an index:

```
map<int, list<Employee>::iterator> employeeIndex;
```

4. Insert at least five Springfield employees into the list.
5. Build the map so that each `eid` is associated with the iterator identifying the corresponding employee in the list.
6. Write a function that receives an `eid` and uses the map to find and display the corresponding employee **without searching the list sequentially**.
7. Write a function to display all employees by traversing the list.
8. Write a function to remove an employee given its `eid`. The function must:
 - Find the employee through the map.
 - Erase the employee from the list.
 - Remove the corresponding entry from the map.
9. Demonstrate that attempting to find an employee whose `eid` does not exist is handled appropriately.
10. Explain why using an iterator in the map is preferable to storing a C++ reference to the employee.

Important observation

- The map is an **index**, not a second copy of the employee database. There is only one `Employee` object for each employee; the map merely provides a fast way to locate that object in the list.
- A `vector<Employee>` plus a map of pointers/iterators would be problematic because inserting into a vector can invalidate pointers/iterators. A list is much better here because **inserting or removing other list elements does not invalidate iterators to existing elements**.
- Caution - when an employee is **removed** from the list, the corresponding map entry must also be removed. Otherwise, the map will contain an invalid iterator.

Solution

```
//=====
// Springfield Employee Database
//
// The list is the primary data structure.
// The map is an index into the list.
//
// map<int, list<Employee>::iterator>
//=====

#include "utility.h"

//=====
// Employee
//=====

struct Employee
{
    int    eid;
    string ename;

    Employee(int id = 0 , string name = "na")
    {
        eid = id;
        ename = name;
    }

    friend ostream& operator<< (ostream& sout, const Employee& e) {
        sout << "[" << e.eid << ", " << e.ename << "]" << endl;
        return sout;
    }
};

//=====
// Display all employees in the list
//=====

void displayEmployees(const list<Employee>& lstEmployees)
{
    cout << "\nEmployees in the list\n";
    cout << "-----\n";

    for (const Employee& employee : lstEmployees)
    {
        cout << employee << endl;
    }
}
```

```

//=====
// Build the map index
//=====

void buildIndex(
    list<Employee>& lstEmployees,
    map<int, list<Employee>::iterator>& mEmployeeIndex)
{
    for (auto it = lstEmployees.begin(); it != lstEmployees.end(); ++it)
    {
        mEmployeeIndex[it->eid] = it;
    }
}

//=====
// Find an employee using the map
//=====

void findEmployee(
    int eid,
    const map<int, list<Employee>::iterator>& mEmployeeIndex)
{
    auto found = mEmployeeIndex.find(eid);

    if (found == mEmployeeIndex.end())
    {
        cout << "Employee " << eid << " was not found.\n";
        return;
    }

    // found->second is an iterator into the list
    cout << *(found->second) << endl;
}

//=====
// Remove an employee
//=====

void removeEmployee(
    int eid,
    list<Employee>& lstEmployees,
    map<int, list<Employee>::iterator>& mEmployeeIndex)
{
    auto found = mEmployeeIndex.find(eid);

    if (found == mEmployeeIndex.end())
    {
        cout << "Employee " << eid << " was not found.\n";
        return;
    }

    // Save the list iterator.
    auto lstEmployeeIterator = found->second;

```

```

        // Remove the entry from the list.
        lstEmployees.erase(lstEmployeeIterator);

        // Remove the corresponding entry from the index.
        mEmployeeIndex.erase(found);
    }

//=====
// Main
//=====

int main()
{
    // The list is the primary employee database.

    list<Employee> lstEmployees;

    lstEmployees.push_back(Employee(101, "Homer Simpson"));
    lstEmployees.push_back(Employee(102, "Marge Simpson"));
    lstEmployees.push_back(Employee(103, "Bart Simpson"));
    lstEmployees.push_back(Employee(104, "Lisa Simpson"));
    lstEmployees.push_back(Employee(105, "Ned Flanders"));
    lstEmployees.emplace_back(100, "Princ. Skinner");

    // The map is an index into the list.
    map<int, list<Employee>::iterator> mEmployeeIndex;

    // Build the index.
    buildIndex(lstEmployees, mEmployeeIndex);

    // Display the entire database.
    displayEmployees(lstEmployees);

    // Find employees using the index.
    cout << "\nSearching for employee 103:\n";
    findEmployee(103, mEmployeeIndex);

    cout << "\nSearching for employee 105:\n";
    findEmployee(105, mEmployeeIndex);

    cout << "\nSearching for employee 999:\n";
    findEmployee(999, mEmployeeIndex);

    // Remove an employee.
    cout << "\nRemoving employee 103...\n";

    removeEmployee(103, lstEmployees, mEmployeeIndex);

    // Display the list after deletion.
    displayEmployees(lstEmployees);
}

```

```

    // Verify that 103 can no longer be found.

    cout << "\nSearching for employee 103 again:\n";
    findEmployee(103, mEmployeeIndex);

    return 0;
}

```

Sample Output

```

Employees in the list
-----
[ 101, Homer Simpson]
[ 102, Marge Simpson]
[ 103, Bart Simpson]
[ 104, Lisa Simpson]
[ 105, Ned Flanders]

Searching for employee 103:
[ 103, Bart Simpson]

Searching for employee 105:
[ 105, Ned Flanders]

Searching for employee 999:
Employee 999 was not found.

Removing employee 103...

Employees in the list
-----
[ 101, Homer Simpson]
[ 102, Marge Simpson]
[ 104, Lisa Simpson]
[ 105, Ned Flanders]

Searching for employee 103 again:
Employee 103 was not found.

```

Utility Method – Use this method to display a generic list

Place this code into the **utility.h** compilation unit. Then use `#include "utility.h"` to import.

```
//-----  
// Utility: showList  
//  
// Prints an optional caption followed by all elements  
// of a list<T> on the console.  
//  
// Parameters:  
//   lst      -- the list to display (passed by const reference  
//               to avoid an unnecessary copy)  
//   caption   -- optional heading printed before the elements  
//   newline   -- true  : each element appears on its own line  
//               false : all elements appear on a single line  
//                 separated by spaces  
//  
// Requires: T must support operator<<  
//-----  
// YOUR TURN: Improve the definition of this function  
//             Use your debugger to visualize the code execution  
//             before and after your changes.  
//             Hint: const reference parameter is a better choice!  
//-----  
template <class T>  
void showList(const list<T>& lst,  
             const string& caption = "",  
             bool newline = false)  
{  
    char terminator = newline ? '\n' : ' ';  
  
    cout << "\n" << caption  
         << " (size=" << lst.size() << ")\n"  
         << string(40, '-') << "\n";  
  
    for (const auto& item : lst)           // improve this fragment!  
        cout << item << terminator;  
  
    cout << "\n";  
}
```